

# Informe técnico — Cooper | CODEFEST AD ASTRA 2026

**Entrega final reproducible, Windows + CUDA.**

Objetivo: recuperación vectorial sobre el corpus oficial sin modelos generativos. El sistema usa un encoder público de Hugging Face, FAISS y metadata JSONL; no utiliza LLM, GPT/Claude/Gemini/Llama, query expansion, agentes, memoria, BM25, ChromaDB ni cross-encoder.

Construcción final: 213,241 vectores / 1,762 documentos con chunks. Encoder: **intfloat/multilingual-e5-base**, dimensión 768. Hardware de construcción: NVIDIA GeForce RTX 3050 Ti Laptop GPU con PyTorch CUDA; dtype de inferencia: float16.

La entrega contiene resultados.jsonl, generador.py, base\_vectorial e informe técnico. Las 50 consultas se conservan en orden y cada una devuelve tres documentos diferentes y diez fragmentos.

## Extracción estructurada y cobertura

PDF: PyMuPDF extrae bloques ordenados por página y elimina únicamente encabezados/pies repetidos. No se concatena el documento completo. JSON: los campos textuales se conservan individualmente con `json_path`. CSV/XLSX: cada fila mantiene pares columna: valor. HTML: párrafos y elementos de lista. PBF: atributos por feature.

El chunker trata listas como ítems completos (incluye líneas envueltas) y tablas/filas como unidades completas. La segmentación de prosa ES/EN/PT solo divide después de puntuación terminal. No inventa fronteras.

Cobertura final: 1,762 documentos con chunks. Las cifras de archivos excluidos y cobertura parcial se toman del registro de fallas generado durante la construcción.

Las imágenes se procesan mediante OCR únicamente cuando se activa explícitamente `--enable-ocr`. Los archivos omitidos quedan registrados en `extraction_failures.jsonl` para conservar trazabilidad.

## Embeddings, límites y caché

Se usa el mismo modelo multilingual-e5-base para pasajes y consultas, con prefijos exactos passage: y query:. Todos los embeddings se normalizan L2 antes de indexar o buscar. El dispositivo se elige explícitamente; --device cuda falla si CUDA no está disponible, y auto cae claramente a CPU.

Batch solicitado/efectivo: 16. Ante CUDA OOM el encoder reduce progresivamente el batch y reintentará; en la construcción final no hubo OOM. float16 se usa solo en inferencia CUDA.

Límite de indexación: objetivo 360 tokens y máximo 480 tokens. El límite de 250 palabras no limita el índice: aplica exclusivamente a resultados.jsonl. Para salida, cada fragmento se parte solo por oraciones o unidades estructurales completas; una unidad realmente indivisible >250 palabras se omite de presentación y se busca el siguiente candidato.

Caché persistente por hash: extraction cache guarda payloads por SHA-256 y evita reabrir fuentes sin cambios; embedding cache SQLite guarda vectores float32 normalizados por modelo/tipo/contenido. Esto hace las verificaciones posteriores reproducibles y evita reextraer/reembeddar el corpus.

## FAISS, recuperación y métricas

Índice: **faiss.IndexFlatIP**, escrito con `faiss.write_index()`. Con L2, el producto interno equivale a similitud coseno exacta. `metadata.jsonl` tiene una línea por vector en el mismo orden; el cargador verifica `index.ntotal` y la dimensión/configuración/prefijos antes de consultar.

Tiempo medido de indexación (embeddings + FAISS): 64.68 s. Tamaño `index.faiss`: 624.7 MiB. Metadata: 213,241 líneas. Resultados generados: 50 consultas.

Se recuperan hasta 1,000 candidatos para asegurar diez salidas que respeten el límite de 250 palabras; la agregación de documentos usa max pooling y devuelve tres `doc_id` distintos. Los diez fragmentos conservan `chunk_id` para trazabilidad.

Las fallas y omisiones se conservan en `extraction_failures.jsonl` y no se ocultan mediante cortes arbitrarios. Los valores de esta ejecución se calculan desde el índice y el registro generado.

## Validación, errores restantes y reproducción

Las pruebas unitarias cubren selección CUDA, normalización L2, correspondencia FAISS-metadata, forma de resultados, listas completas, extracción JSON y caché de extracción. `gpu_smoke_test` valida Python, PyTorch/CUDA, dimensión, embeddings ES/EN/PT y búsqueda FAISS mínima.

Preflight de entrega valida desde cero carga FAISS, igualdad índice/metadata, esquema metadata, 50 queries ordenadas, 3 documentos, 10 fragmentos, ids existentes y máximo de 250 palabras. `generador.py` vuelve a cargar el índice persistido y reproduce `resultados.jsonl`.

Errores registrados en esta ejecución: 400. El detalle por tipo se conserva en `extraction_failures.jsonl`. No se introdujo corte arbitrario para ocultar unidades que no cumplen los límites.

Reproducción en Windows: activar `.venv`, instalar dependencias y wheel CUDA de PyTorch, ejecutar `scripts/gpu_smoke_test.py --device cuda`, construir con `scripts/build_baseline.py`, y ejecutar `generador.py`. FAISS se usa en CPU de forma compatible; CUDA acelera los embeddings.